Hello, Python!

Course: Kaggle Learn · Instructor: Colin Morris · Lesson: Hello, Python!

1. Introduction to Python

Python is a programming language widely used in data science, automation, web development, and many other fields. It is known for its readable syntax and relatively simple structure.

The Kaggle Learn Python course introduces essential Python skills for learners who already have some programming experience and want to use Python for data science.

This lesson covers:

- Variable assignment and reassignment.
- The `print()` and `type()` built-in functions.
- Comments, strings, and indentation.
- Numbers and arithmetic operators.
- Operator precedence and parentheses.
- Built-in functions for working with numbers.
- Converting values between numeric types.

2. Variables and Variable Assignment

2.1 What Is a Variable?

A variable is a name that refers to a value. Variables allow us to store information and use it later in a program.

In Python, we create a variable using the assignment operator (`=`).

Example:

```
spam_amount = 0
print(spam_amount)
```

Output:

```
0
```

Explanation:

- `spam_amount` is the variable name.
- `=` is the assignment operator.
- `0` is the value assigned to the variable.
- `print(spam_amount)` displays the variable's current value.

2.2 Variable Assignment in Python

Python does not require us to declare a variable before assigning a value to it. We also do not need to specify its data type explicitly.

```
age = 19
name = "Aru"
is_student = True
```

Here, Python automatically determines the type of each value.

Variable	Value	Data type
age	19	int
name	"Aru"	str
is_student	True	bool

2.3 Variable Reassignment

We can change the value associated with an existing variable by assigning a new value to it.

```
spam_amount = 0
spam_amount = spam_amount + 4 # Increase the value by 4
print(spam_amount)
```

Output:

```
4
```

How it works:

1. Initially, `spam_amount` refers to `0`.
2. Python evaluates the right-hand expression: `spam_amount + 4`.
3. The result is `0 + 4 = 4`.
4. The assignment operator stores the result in `spam_amount`.

The same variable can also refer to a value of a different type.

```
value = 10
value = "Hello"
value = True
print(value)
```

Output:

```
True
```

Important: The `=` operator assigns a value. It does not mean that the two sides are mathematically equal.

3. Displaying Output and Using Comments

3.1 The `print()` Function

The `print()` function displays information in the output.

Syntax:

```
print(value)
```

Examples:

```
print(100)
print("Hello, Python!")
print(5 + 3)
```

Output:

```
100
Hello, Python!
8
```

Python evaluates expressions before displaying their results.

Functions are called by writing their name followed by parentheses. The values supplied inside the parentheses are called arguments.

3.2 Comments in Python

Comments explain code and are ignored during normal program execution.

A single-line comment begins with the `#` symbol.

```
spam_amount = 0                # Store the initial spam amount
spam_amount = spam_amount + 4  # Increase the spam amount
print(spam_amount)
```

Output:

```
4
```

Comments help make programs easier to understand, maintain, and revise.

3.3 Printing Multiple Values

The `print()` function can display multiple arguments separated by commas.

```
height = 2.15
print("Height in meters =", height, "m")
```

Output:

```
Height in meters = 2.15 m
```

By default, `print()` separates multiple arguments with spaces.

4. Conditional Statements and Indentation

4.1 The `if` Statement

An `if` statement executes a block of code only when its condition evaluates to `True`.

Syntax:

```
if condition:    # Code executed when the condition is True
```

Example:

```
spam_amount = 4
if spam_amount > 0:
   print("But I don't want ANY spam!")
```

Output:

```
But I don’t want ANY spam!
```

Explanation:

- `spam_amount > 0` checks whether the value is greater than zero.
- The colon (`:`) marks the beginning of the conditional code block.
- The indented `print()` statement executes because the condition is `True`.

If `spam_amount` were `0`, the message would not be printed.

4.2 Indentation and Code Blocks

Python uses indentation to determine which statements belong to a code block.

```
spam_amount = 4
if spam_amount > 0:
   print("Spam detected")
print("Program continues")
```

Output:

```
Spam detected
Program continues
```

The first `print()` statement is indented and belongs to the `if` block. The second statement is not indented, so it executes regardless of the condition.

Important rules:

- A colon (`:`) introduces the `if` block.
- Statements inside the block must be indented consistently.
- Four spaces are the conventional indentation standard.
- Indentation affects program behavior; it is not merely decorative.

Unlike languages such as C++ and Java, Python generally uses indentation rather than curly braces (`{}`) to define code blocks.

5. Strings and String Operations

5.1 What Is a String?

A string is a sequence of characters used to represent text.

Strings can be enclosed in either single or double quotation marks.

```
name = "Python"
message = 'Hello, World!'
print(name)
print(message)
```

Output:

```
Python
Hello, World!
```

Both quotation styles are valid. Choose the one that makes the string easiest to write.

For example, double quotes are convenient when the string contains an apostrophe:

```
message = "But I don't want ANY spam!"
print(message)
```

Output:

```
But I don't want ANY spam!
```

Using single quotes around this particular string would require escaping the apostrophe or choosing another valid quoting style.

5.2 String Multiplication

Python allows a string to be multiplied by an integer. This repeats the string the specified number of times.

Syntax:

string * integer

Example:

viking_song = "Spam " * 4print(viking_song)

Output:

Spam Spam Spam Spam

Explanation:

- `"Spam "` is the string being repeated.
- `4` specifies the number of repetitions.
- The trailing space in `"Spam "` is repeated too.

String repetition is useful for generating repeated text without writing the same string multiple times.

5.3 Operator Overloading

Python operators can behave differently depending on the types of values they operate on. This is called operator overloading.

Operator	With numbers	With strings
`+`	Adds numbers	Concatenates strings
`*`	Multiplies numbers	Repeats a string when multiplied by an integer

Examples:

```
print(3 + 4)
print("Spam" + " eggs")
print(3 * 4)
print("Ha" * 3)
```

Output:

```
7
Spam eggs
12
HaHaHa
```

The same operator can therefore perform different operations depending on its operands.

6. Numbers and Arithmetic Operators

Python provides several numeric types. This lesson introduces integers and floating-point numbers.

6.1 Integers (`int`)

An integer is a whole number without a fractional or decimal component.

```
spam_amount = 0
print(type(spam_amount))
```

Output:

```
<class 'int'>
```

Examples of integer values include:

```
19
-25
1000
```

6.2 Floating-Point Numbers (`float`)

A floating-point number represents a number that can contain a fractional component.

```
print(type(19.95))
```

Output:

```
<class 'float'>
```

Examples:

```
weight = 72.5
temperature = 36.6
percentage = 0.75
```

Floats are useful for representing quantities such as measurements, weights, and proportions.

6.3 Arithmetic Operators

Python supports the following basic arithmetic operators.

Operator	Name	Description	Example	Result
`+`	Addition	Sum of two values	`5 + 2`	`7`
`-`	Subtraction	Difference between values	`5 - 2`	`3`
`*`	Multiplication	Product of values	`5 * 2`	`10`
`/`	True division	Division that returns a float	`5 / 2`	`2.5`
`//`	Floor division	Division rounded down to an integer boundary	`5 // 2`	`2`
`%`	Modulus	Remainder after division	`5 % 2`	`1`
`**`	Exponentiation	Raises a value to a power	`5 ** 2`	`25`
`-a`	Negation	Reverses a number's sign	`-5`	`-5`

6.4 True Division (`/`)

True division uses `/` and returns a floating-point result, even when the result is mathematically a whole number.

```
print(5 / 2)
print(6 / 2)
```

Output:

```
2.5
3.0
```

Notice that `6 / 2` produces `3.0`, not `3`.

6.5 Floor Division (//)

Floor division uses // and rounds the quotient down to the nearest integer boundary.

```
print(5 // 2)
print(6 // 2)
```

Output:

```
2
3
```

Floor division can be useful when working with whole groups, such as determining how many complete pairs can be formed from a collection.

For example:

```
people = 17
people_per_group = 4
complete_groups = people // people_per_group
print(complete_groups)
```

Output:

```
4
```

There are four complete groups of four people, with one person left over.

Important caveat: Floor division means rounding toward negative infinity, not simply removing the decimal portion in every case.

```
print(-5 // 2)
```

Output:

```
-3
```

6.6 Modulus (%)

The modulus operator returns the remainder after division.

```
print(5 % 2)
print(10 % 3)
```

Output:

```
1
1
```

It is useful for checking whether a number is divisible by another number.

```
number = 8
print(number % 2)
```

Output:

```
0
```

A remainder of `0` means that `number` is evenly divisible by `2`.

6.7 Exponentiation (**)

The exponentiation operator raises a number to a specified power.

```
print(2 ** 3)
print(5 ** 2)
```

Output:

```
8
25
```

Here, `2 ** 3` means $2^3 = 2 \times 2 \times 2$.

7. Order of Operations and Parentheses

7.1 Operator Precedence

When an expression contains multiple operators, Python follows rules that determine the order in which operations are evaluated.

A useful simplified version of the familiar PEMDAS rule is:

1. Parentheses
2. Exponentiation
3. Multiplication, division, floor division, and modulus
4. Addition and subtraction

Operators at the same precedence level are generally evaluated from left to right, except for exponentiation, which groups from right to left.

Examples:

```
print(8 - 3 + 2)
print(-3 + 4 * 2)
```

Output:

```
7
5
```

Explanation:

- `8 - 3 + 2` is evaluated from left to right: `5 + 2 = 7`.
- In `-3 + 4 * 2`, multiplication happens first: `-3 + 8 = 5`.

7.2 Using Parentheses to Control Evaluation

Parentheses allow us to explicitly specify which part of an expression should be evaluated first.

Consider the following example:

```
hat_height_cm = 25
my_height_cm = 190     # Calculate total height in meters
total_height_meters = hat_height_cm + my_height_cm / 100
print("Height in meters =", total_height_meters, "?")
```

Output:

```
Height in meters = 26.9 ?
```

Why is the result incorrect?

Python performs division before addition:

1. `190 / 100` gives `1.9`.
2. `25 + 1.9` gives `26.9`.

The hat height is also measured in centimeters, so both heights must be added before converting the total to meters.

Correct version:

```
hat_height_cm = 25
my_height_cm = 190
```

```
total_height_meters = (hat_height_cm + my_height_cm) / 100
print("Height in meters =", total_height_meters)
```

Output:

```
Height in meters = 2.15
```

Explanation:

1. Parentheses calculate `25 + 190 = 215`.
2. Division converts `215` centimeters to `2.15` meters.

Remember: Use parentheses whenever the intended order of calculations needs to be made explicit.

8. Built-in Functions for Working with Numbers

Python includes built-in functions that simplify common numerical operations.

8.1 `type()`

Returns the type of an object.

Syntax:

type(value)

Example:

```
print(type(10))
print(type(19.95))
print(type("Python"))
```

Output:

```
<class 'int'>
<class 'float'>
<class 'str'>
```

Use `type()` when you need to identify the type of a value.

8.2 `min()`

Returns the smallest value among its arguments.

Syntax:

min(value1, value2, ...)

Example:

```
print(min(1, 2, 3))
print(min(10, -5, 7))
```

Output:

```
1
-5
```

8.3 `max()`

Returns the largest value among its arguments.

Syntax:

max(value1, value2, ...)

Example:

```
print(max(1, 2, 3))
print(max(10, -5, 7))
```

Output:

```
3
10
```

8.4 `abs()`

Returns the absolute value of a number, representing its non-negative magnitude.

Syntax:

```
abs(number)
```

Example:

```
print(abs(32))
print(abs(-32))
print(abs(0))
```

Output:

```
32
32
0
```

Absolute value removes the negative sign when a number is negative.

8.5 `float()`

Converts a compatible value into a floating-point number.

Syntax:

float(value)

Example:

```
print(float(10))
```

Output:

```
10.0
```

An integer becomes a float.

8.6 `int()`

Converts a compatible value into an integer.

Syntax:

int(value)

Example:

```
print(int(3.33))
print(int(-3.33))
```

Output:

```
3
-3
```

When converting a float, `int()` truncates the fractional part toward zero; it does not round to the nearest integer.

8.7 Converting Strings to Numbers

The `int()` function can convert a string containing a valid integer representation into an integer.

```
print(int('807') + 1)
```

Output:

```
808
```

Explanation:

1. `'807'` is a string.
2. `int('807')` converts it to the integer `807`.
3. `807 + 1` evaluates to `808`.

A string containing a decimal representation, such as `'3.33'`, cannot be converted directly using `int()`. For that, convert it to a float first if appropriate.

```
print(int(float('3.33')))
```

Output:

```
3
```

9. Practical Example: The Spam Program

The following program combines variable assignment, reassignment, comments, conditionals, strings, arithmetic, and output.

```
# Start with zero servings of spam
spam_amount = 0
print(spam_amount)     # Order four more servings
spam_amount = spam_amount + 4       # Display a message if the amount is positive
if spam_amount > 0:
   print("But I don't want ANY spam!")     # Repeat the string according to the amount
   viking_song = "Spam " * spam_amount
print(viking_song)
```

Output:

```
0
But I don't want ANY spam!
Spam Spam Spam Spam
```

Step-by-step execution

1. `spam_amount = 0` assigns the initial value.
2. `print(spam_amount)` displays `0`.
3. `spam_amount = spam_amount + 4` changes the value to `4`.
4. `if spam_amount > 0:` checks whether the value is positive.
5. The indented `print()` statement executes because the condition is true.
6. `"Spam " * spam_amount` repeats the string four times.
7. The final `print()` displays the repeated string.

This example demonstrates how Python combines several basic programming concepts in a single program.

10. Quick Reference

10.1 Variables and Syntax

Syntax	Purpose
`x = 10`	Assign a value to a variable
`x = x + 1`	Reassign a variable using its previous value
`print(x)`	Display a value
`type(x)`	Identify a value's type
`# comment`	Add a comment
`if x > 0:`	Begin a conditional block

`name = "Python"`	Create a string
`"Ha" * 3`	Repeat a string

10.2 Arithmetic Operators

Operator	Meaning	Example result
`+`	Addition	`4 + 2` → `6`
`-`	Subtraction	`4 - 2` → `2`
`*`	Multiplication	`4 * 2` → `8`
`/`	True division	`5 / 2` → `2.5`
`//`	Floor division	`5 // 2` → `2`
`%`	Remainder	`5 % 2` → `1`
`**`	Exponentiation	`4 ** 2` → `16`
`-x`	Negation	`-4` → `-4`

10.3 Built-in Functions

Function	Purpose	Example
`print()`	Display output	`print(10)`
`type()`	Return an object's type	`type(10)`
`min()`	Find the minimum	`min(2, 5, 1)` → `1`
`max()`	Find the maximum	`max(2, 5, 1)` → `5`
`abs()`	Return absolute value	`abs(-8)` → `8`
`int()`	Convert to an integer	`int(3.9)` → `3`
`float()`	Convert to a float	`float(10)` → `10.0`

10.4 Key Concepts

- `=` is the assignment operator.
- Python does not require explicit variable declarations or type annotations for basic assignment.
- `int` represents integers; `float` represents floating-point numbers.
- Strings can use single or double quotes.
- The `*` operator can repeat strings when paired with an integer.
- Python uses indentation to define code blocks.
- `/` performs true division, while `//` performs floor division.
- `%` returns a remainder.
- Parentheses control the order of evaluation.
- `int()` truncates a float toward zero rather than rounding it.
- `min()`, `max()`, `abs()`, `int()`, `float()`, `type()`, and `print()` are built-in functions.

11. Common Mistakes to Remember

1. Mistake: Confusing assignment (`=`) with equality comparison.
    - Correction: `=` assigns a value. `==` compares two values for equality.
2. Mistake: Forgetting the colon after an `if` statement.
    - Correction: Write `if condition:` before the indented block.
3. Mistake: Using inconsistent indentation.
    - Correction: Indent statements within a block consistently, conventionally using four spaces.
4. Mistake: Assuming `/` always returns an integer when dividing evenly.
    - Correction: `/` returns a float, such as `6 / 2` producing `3.0`.
5. Mistake: Assuming `//` simply removes the decimal part.
    - Correction: Floor division rounds down toward negative infinity. For example, `-5 // 2` produces `-3`.
6. Mistake: Assuming `int(3.9)` rounds to `4`.
    - Correction: `int(3.9)` truncates the fractional part and produces `3`.
7. Mistake: Forgetting operator precedence.
    - Correction: Use parentheses to make the intended calculation order explicit.
8. Mistake: Trying to perform arithmetic directly on numeric strings.
    - Correction: Convert strings to an appropriate numeric type first, such as `int('807') + 1`.
9. Mistake: Assuming operators always behave the same way for every data type.
    - Correction: Remember operator overloading: `+` adds numbers or concatenates strings, while `*` multiplies numbers or repeats strings.
10. Mistake: Including quotation marks when you want to use a variable's value.
    - Correction: `print(name)` displays the value stored in `name`, while `print("name")` displays the literal text `name`.

Next step: Complete the Kaggle Learn Hello, Python! exercise to practise variable assignment, arithmetic operators, and Python syntax.